FPT UNIVERSITY

MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

DPL Final Project Report

Single-Image 2D-to-3D Reconstruction with Transformers

Group 8	
Group Members	Đỗ Nguyễn Hữu Phúc - SE204297 Nguyễn Tiến Vinh - SE204328 Phạm Trường Minh Nhật - SE205274
Supervisor	Nguyễn Xuân Huy
Course / Topic	DPL - Topic 3: 2D → 3D reconstruction with Transformers

- Ho Chi Minh City, 11/2026 -

Table of Contents

Acknowledgement....3
Definition and Acronyms....3
List of tables....3
List of figures....3
I. Introduction....5
1. Project Information....5
2. Motivation....5
3. Problem Statement....5
4. Objectives and Scope....5
5. Main Contributions....5
II. Related Work....6
1. Single-Image 3D Reconstruction....6
2. Transformers for 3D Reconstruction....6
3. Datasets and Metrics....6
4. Gaps and Choice of Baselines....6
III. Data....7
1. Sources....7
2. Build Pipeline....7
3. Statistics and Class Balance....7
4. Split, Leakage and Cleaning....8
5. Preprocessing and Augmentation....8
IV. Baselines....9
1. Baseline 1: RGB2Point....9
2. Baseline 2: 3D-RETR-S....9
V. Proposed Method....10
1. Motivation from Error Analysis....10
2. Point Cloud Line (Baseline 1 → V0-V3)....10
3. Voxel Line (Baseline 2 → V1-V4)....11
4. Losses and Training Strategy....11
VI. Experiments....12
1. Settings and Hyper-parameters....12
2. Main Results....12
3. Ablation Study....13
4. Cross-Representation Comparison....13
5. Error Analysis....14
5.1 Viewpoint vs object....14
5.2 Mean-shape collapse and density floor....14
5.3 Per-object comparison....14
5.4 Attention analysis of B1-V1....15
5.5 Voxel errors and best/worst cases....15
6. Qualitative Results on Real Photos....16
7. Efficiency....17
VII. Demo and Deployment....18
1. Application Architecture....18
2. Inference Pipeline....18
3. User Guide....18
4. REST API (FastAPI)....18
5. Docker, CI/CD and Cloud Deployment....19
VIII. Conclusion and Future Work....20
1. Summary of Findings....20
2. Strengths and Weaknesses....20
3. Limitations....20
4. Future Work....20
IX. References....20
X. Appendices....21
A. Task Assignment....21
B. Use of AI Tools....21

C. Reproducibility.....21

Acknowledgement

We thank our lecturer, Mr. Nguyễn Xuân Huy, for his guidance on the choice of baselines, the experimental protocol and the structure of this project, and the authors of RGB2Point, 3D-RETR, ShapeNet, 3D-R2N2, 3D-LMNet and Pix3D for releasing their code and data.

Definition and Acronyms

Table 1: Acronyms used in this report

Acronym	Definition
CD / EMD	Chamfer Distance / Earth Mover's Distance (both reported ×100, 1024 points)
F@0.01	F-score at distance threshold 0.01
IoU	Intersection over Union of 32^3 voxel grids (threshold 0.5)
GT	Ground truth
ViT / DeiT	Vision Transformer / Data-efficient image Transformer
DINOv2 (reg4)	Self-supervised ViT features (Meta AI); reg4 = with 4 register tokens
CLS	Class token of a ViT (one global image vector)
CFI	Contextual Feature Integrator (self-attention block of RGB2Point)
R2N2	3D-R2N2 renderings and voxels of ShapeNet
bf16	bfloat16 mixed precision
API / CI/CD	Application Programming Interface / Continuous Integration and Deployment
GHCR / HF	GitHub Container Registry / Hugging Face
V0..V4	Our improved versions of a baseline, one change at a time

List of tables

Table 1: Acronyms used in this report.....3
Table 2: Data sources (all offline downloads).....7
Table 3: Dataset statistics (per object; images = objects × views).....7
Table 4: Baseline 1: paper vs ours (CD / EMD ×100 / F@0.01).....9
Table 5: Baselines, paper vs ours (means over airplane, car, chair).....9
Table 6: Hyper-parameters of every run.....12
Table 7: Point cloud line, full test set (CD / EMD ×100 / F@0.01; view 00, 3668 objects; CD, EMD, F = mean over classes; Train = trainable parameters).....12
Table 8: Voxel line, full test set (IoU; view 00, 3668 objects).....12
Table 9: Ablation: one change per version (CD ×100 lower is better, IoU higher is better).....13
Table 10: All models on common metrics (test view 00, means over 3 classes).....14
Table 11: Diversity of predictions: metric between two different objects (higher CD / lower IoU = more diverse).....14
Table 12: Baseline 2 by GT size quartile (Q0 = thinnest objects); IoU / precision / recall.....15
Table 13: Size and speed of the models used in the demo.....17
Table 14: Task assignment.....21

List of figures

Figure 1: Data build pipeline: sources merged by ShapeNet object ID, checked, then prepared per baseline; Pix3D is a held-out real-image test set.....7
Figure 2: Objects per class and split, and class shares by objects and by images; chair has only 10 views.....8
Figure 3: Model inputs (black background) and GT voxels projected along three axes.....8
Figure 4: Baseline 1: train loss, val CD and learning rate.....9

Figure 5: Baseline 2: Dice loss and val IoU per class....9
Figure 6: (a) RGB2Point baseline. (b) Our point cloud model: CLS + pooled patch tokens, a query-based Transformer decoder, and a DINOv2 encoder (V2 frozen; V3 last 4 blocks fine-tuned)....10
Figure 7: (a) 3D-RETR-S baseline. (b) Our voxel model: DINOv2 ViT-S/14 (reg4) encoder with a 384 → 192 projection; decoder and 3D CNN unchanged (V2: 8^3 query grid)....11
Figure 8: Validation curves of every version: point cloud line (left; V3 continues from V2 for 8 epochs) and voxel line (right)....13
Figure 9: Ablation on ShapeNet test and Pix3D chair: point cloud line CD (left), voxel line IoU (right). V0 was not evaluated on Pix3D....13
Figure 10: Per-object results on test chairs (mean over views): B1 base vs V1 CD, V1 vs V2 CD, B2 base vs V1 IoU. Points below (CD) or above (IoU) the red line favour the newer version....14
Figure 11: B1-V1 cross-attention on two test chairs. Layer 1: queries of the lowest 10% of points look at legs and base, the top 10% at the backrest. Layer 4: all queries look at the same background cells....15
Figure 12: Best and worst test objects per class. Left, Baseline 1: the worst airplane has an abnormally small GT, the worst chair thin legs. Right, Baseline 2: the worst chair is a mis-oriented GT, the worst car a tractor. 16
Figure 13: Pix3D chairs, point cloud line: baseline, V2, V3, GT. V3 recovers legs and backrest better; the reclining chair (row 3) fails for all....16
Figure 14: Pix3D chairs, voxel line (side projection): baseline, V3, V4, voxelised GT....16
Figure 15: Internet photos (Wikimedia Commons, test only) through the demo pipeline, 4 models. B1-V3 gives clearer chair legs and backrest than RGB2Point. Photos: "Library Chair dakota jackson.jpg" by Dakota Jackson Inc., CC BY-SA 3.0, commons.wikimedia.org/wiki/File:Library_Chair_dakota_jackson.jpg; "Abandoned car in Marine Park (10852p).jpg" by Rhododendrites, CC BY-SA 4.0, commons.wikimedia.org/wiki/File:Abandoned_car_in_Marine_Park_(10852p).jpg....17
Figure 16: Application and CI/CD architecture. One container serves the Gradio demo (/) and the REST API (/predict, /docs); weights come from the Hugging Face Hub on first use. HF Hub only hosts the weights; the app runs on Cloud Run....18
Figure 17: Upload image tab: a chair photo, the model input after background removal, B1-V3 points and B2-V4 voxels. Photo: "Library Chair dakota jackson.jpg" by Dakota Jackson Inc., CC BY-SA 3.0, commons.wikimedia.org/wiki/File:Library_Chair_dakota_jackson.jpg....18
Figure 18: Test set tab: a test airplane with its ground truth and per-object metrics (CD, F@0.01, IoU) next to the predictions....18
Figure 19: Uploaded photos after background removal and cropping. Photos: Dakota Jackson Inc. (CC BY-SA 3.0) and Rhododendrites (CC BY-SA 4.0), Wikimedia Commons; full credits in Figure 15....19
Figure 20: Swagger page of the API (/docs)....19
Figure 21: GitHub Actions run of the README commit: the four jobs (test, docker, ghcr, deploy) passed.....19
Figure 22: Cloud Run: the image-to-3d service in asia-southeast1....19
Figure 23: GHCR: the public image ghcr.io/ilogicus/image-to-3d-transformer:latest....19

I. Introduction

1. Project Information

- **Group 8:** Đỗ Nguyễn Hữu Phúc (SE204297), Nguyễn Tiến Vinh (SE204328), Phạm Trường Minh Nhật (SE205274). **Supervisor:** Nguyễn Xuân Huy.
- **Code:** github.com/iLogicus/image-to-3d-transformer. **Weights:** huggingface.co/iLogicus/image-to-3d-weights.
- **Public demo:** image-to-3d-98814186060.asia-southeast1.run.app (demo at /, REST API at /docs).
- **Data and checkpoints:** drive.google.com/drive/folders/1a3uJbDSbtzmmrHbJdoV02D4LfoUdznkw.

2. Motivation

Recovering a 3D shape from one photo supports e-commerce previews, AR/VR content, robotics and fast asset creation. The task is ill-posed: hidden parts, depth and scale are not observed, so a model must learn a shape prior per category. Transformers fit this well: a pretrained ViT gives strong image features, and attention lets each part of the 3D decoder read the image regions it needs. We study the problem with two baselines that output different representations, a point cloud and a voxel grid.

3. Problem Statement

- **Input:** one RGB image (224 × 224) of an airplane, car or chair: a ShapeNet rendering or a real photo.
- **Output:** a shape in the object's canonical frame: 1024 points (N × 3) or a 32^3 occupancy grid.
- **Challenges:** occluded parts, thin structures (chair legs), the rendering-to-photo gap, and different coordinate frames across datasets.

4. Objectives and Scope

- Build a leak-free dataset (3 classes, split by object) from public sources.
- Reproduce RGB2Point [1] and 3D-RETR-S [2] on one split, seed and hardware, and explain the gaps to the papers.
- Improve each baseline step by step, each step motivated by error analysis and measured by ablation.
- Test on real photos (Pix3D chair) and deliver a demo, REST API, Docker image, CI/CD and public deployment.
- **Scope:** single view, three classes, low-resolution outputs (as the baselines), at most a few hours of training per run on one L4 GPU; the papers trained far longer (1000 epochs; 1-3 days on 2 GPUs).

5. Main Contributions

- A merged, axis-aligned dataset from four public releases, with a leakage audit against the RGB2Point split.
- Plain-PyTorch re-implementations of both baselines (3D-RETR ported from PyTorch 1.8 / Lightning 1.3), within 6% CD and 0.016-0.047 IoU of the papers.
- Point cloud line: a query-based Transformer decoder over patch tokens, DINOv2 features and partial fine-tuning. Mean CD 3.985 → 3.424 (-14%) on ShapeNet and 6.545 → 5.486 (-16%) on Pix3D.
- Voxel line: DINOv2 encoder and cosine schedule, IoU 0.678 → 0.692; augmentation, Pix3D IoU 0.293 → 0.327.
- Error analysis (viewpoint, mean-shape collapse, density floor, attention) that explains each design step.
- A deployed application: Gradio demo and FastAPI on Google Cloud Run, Docker, a 4-job CI/CD pipeline.

II. Related Work

1. Single-Image 3D Reconstruction

Learning-based methods recover a shape from one image by learning a shape prior per category. Voxel decoders such as 3D-R2N2 [4] predict an occupancy grid, which is easy to train with a per-cell loss but grows cubically in memory; point-set decoders predict a fixed number of points trained with CD or EMD, which is light but has no surface connectivity. Tatarchenko et al. [11] showed that many single-view networks behave like shape retrieval and predict a category-typical shape; we test this for our models in Section VI.5.

2. Transformers for 3D Reconstruction

ViT [7] applies self-attention to 16 × 16 image patches; DeiT [8] trains it efficiently with distillation; DINOv2 [9] learns general features by self-supervision, and its register variant [10] removes high-norm artefact tokens. **3D-RETR** [2] encodes the image with a ViT/DeiT and decodes a 4^3 grid of learned queries that cross-attend to the image tokens; a 3D CNN upsamples them to 32^3 voxels (Dice loss). **RGB2Point** [1] maps the CLS token of a frozen ViT through a self-attention block and an MLP to 1024 points (Chamfer loss).

3. Datasets and Metrics

ShapeNet [3] via the 3D-R2N2 release [4] gives 24 renderings (137 × 137) and 32^3 voxels per object; **3D-LMNet** [5] gives point clouds of the same models and of Pix3D; **Pix3D** [6] gives real furniture photos with aligned CAD models. With P the predicted and Q the GT set (1024 points, diagonal about 1):

$$\mathrm{CD}(P,Q) = \frac{1}{|P|}\sum_{p\in P}\min_{q\in Q}\|p-q\|_2 + \frac{1}{|Q|}\sum_{q\in Q}\min_{p\in P}\|q-p\|_2 \tag{1}$$

$$\mathrm{EMD}(P,Q) = \min_{\phi:P\to Q}\ \frac{1}{|P|}\sum_{p\in P}\|p-\phi(p)\|_2 \tag{2}$$

$$\mathrm{F@}d = \frac{2\,\mathrm{Prec}(d)\,\mathrm{Rec}(d)}{\mathrm{Prec}(d)+\mathrm{Rec}(d)}, \qquad \mathrm{IoU} = \frac{|V_{\mathrm{pred}}\cap V_{\mathrm{gt}}|}{|V_{\mathrm{pred}}\cup V_{\mathrm{gt}}|} \tag{3}$$

Prec(d) and Rec(d) are the shares of predicted and GT points within distance d = 0.01 of the other set; V_pred = {σ(logits) > 0.5}. CD and EMD (exact Hungarian matching) are ×100, lower is better; F and IoU are higher-is-better. We also report parameters, ms/image and GPU memory on the L4. Metrics are per class; means are over the 3 classes.

4. Gaps and Choice of Baselines

- RGB2Point keeps only the CLS token, so its self-attention runs over one token and ignores spatial features.
- 3D-RETR uses a small supervised DeiT and drops sharply on real photos (Pix3D IoU 0.29).
- Neither paper analyses failure modes per object or the real-photo gap.
- Both have public code, fit our budget and cover the two main representations (point cloud and voxel).

III. Data

1. Sources

The Stanford R2N2 server is offline, so the dataset was assembled from the releases in Table 2, stored once on Google Drive and unzipped to the Colab disk each session. ShapeNet-derived data are for non-commercial research; Pix3D is for research. The 3D GT comes with the data, so labelling is replaced by checking and cleaning.

Table 2: Data sources (all offline downloads)

Data	Source	Content used	Samples used	Format	Size
Airplane, car renderings	RGB2Point release (authors' Google Drive)	24 views / object, 137×137	276,960 images	PNG RGBA	3.12 GB
Chair renderings, voxels	HF mirror (byte-identical to R2N2): huggingface.co/datasets/learning3dvision/r2n2_shapenet_dataset_full	chair views 00-09; 32^3 voxels	67,780 images; 18,318 grids	PNG, binvox	13.69 GB
GT point clouds	3D-LMNet ShapeNet_pointclouds.zip	1024 points per object	18,318 objects	.npy	2.77 GB
Real photos	Pix3D (pix3d.csail.mit.edu)	chair images, masks, metadata	2,777 images	JPG/PNG, JSON	3.78 GB
Pix3D point clouds	3D-LMNet pix3d_pointclouds.zip	1024 points per CAD model	chair models	.npy	13.3 MB
Split	3D-RETR data/ShapeNet.json	train / val / test object IDs	18,318 IDs	JSON	-

2. Build Pipeline

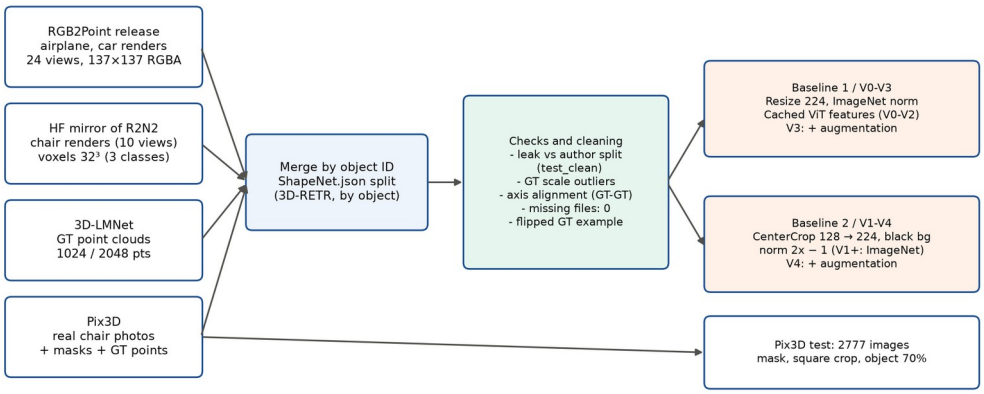


Figure 1: Data build pipeline: sources merged by ShapeNet object ID, checked, then prepared per baseline; Pix3D is a held-out real-image test set.

Objects are joined by ShapeNet ID; all 3668 test objects have renderings, points and voxels. Axis conventions were measured, not assumed: among candidate permutations we kept the one with the lowest CD between surface-sampled GT voxels and GT points (CD ×100 3.85-3.91). The same GT-to-GT search chose the Pix3D-to-ShapeNet rotation.

3. Statistics and Class Balance

Table 3: Dataset statistics (per object; images = objects × views)

Class	Train	Val	Test	Objects	% obj.	Views	Images	% img.	Occ. voxels
airplane	2831	404	810	4045	22.1	24	97,080	28.2	2.02%
car	5245	749	1501	7495	40.9	24	179,880	52.2	9.22%
chair	4744	677	1357	6778	37.0	10	67,780	19.7	11.89%
Total	**12,820**	**1,830**	**3,668**	**18,318**	**100**	**-**	**344,740**	**100**	**-**

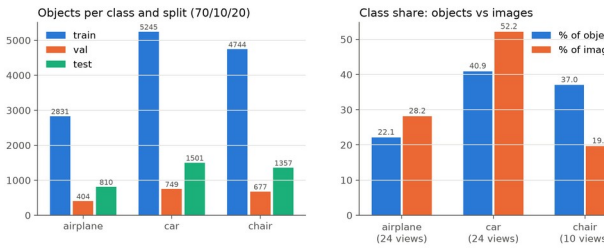


Figure 2: Objects per class and split, and class shares by objects and by images; chair has only 10 views.

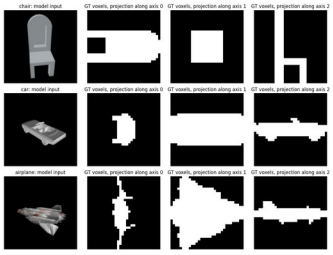


Figure 3: Model inputs (black background) and GT voxels projected along three axes.

Imbalance. Car dominates by images (52.2% vs chair 19.7%) because the chair mirror has 10 views. Baseline 1 trains on every view, so chair gets the fewest steps; Baseline 2 samples one view per object per epoch, which balances by object: Figure 2 (right) shows the class shares before (by images) and after (by objects) this sampling. As this is regression, we report every metric per class instead of re-weighting.

4. Split, Leakage and Cleaning

- **Split:** 3D-RETR object lists (70/10/20), all views of an object in one split. We kept this standard split instead of 70/15/15 so that results stay comparable with the papers. Selection on val only; test uses view 00 (as both codes), error analysis uses all views.
- **Leakage:** 582/810 airplane, 521/1501 car and 901/1357 chair test objects are in the RGB2Point authors' training set. **test_clean** (1664 images) drops them for a fair check of the released weights.
- **Outliers:** 12 airplane and 4 chair test objects have a GT diagonal < 0.8 (median 0.99); kept, they appear as worst cases (Figure 12). Some R2N2 chairs have an upside-down GT (for example test chair 19ee0ca9).
- **Viewpoints:** elevation 25-30°, azimuth 360°; photos outside this range are harder.

5. Preprocessing and Augmentation

- **Baseline 1, V0-V3:** RGBA → RGB, resize 224, ImageNet mean/std; target 1024 centred GT points.
- **Baseline 2:** CenterCrop 128 of 137, resize 224, black background, 2x − 1 (official); target binvox 32^3. V1-V4 use ImageNet mean/std for DINOv2.
- **Pix3D and demo uploads:** object mask (Pix3D mask or rembg U2-Net), square crop so the object fills 70% of the side, background as in training; Pix3D GT points are rotated to ShapeNet axes and voxelised for IoU.

The official 3D-RETR command leaves the background black; we kept it. Frozen-ViT features (B1, V0-V2) are cached on Drive, so an epoch over 241k views takes 100-220 s. **Augmentation** (B1-V3, B2-V4 only): RandomAffine (translate 5%, scale 0.9-1.1) on the RGBA image, a random-colour background, then ColorJitter(0.3, 0.3, 0.3). No flips or rotations, since the GT is in the canonical frame and would no longer match.

IV. Baselines

Both baselines share the hardware (L4, bf16), seed 0, learning rate 1e-4, 50 epochs and split; optimiser, loss and batch size follow each original code (Table 6).

1. Baseline 1: RGB2Point

Model. A frozen ImageNet ViT-B/16 gives the CLS token (768-d); Linear to 4096-d, a self-attention block (CFI) on this single token, and an MLP (4096-2048-2048-3072) output 1024 × 3 points (Figure 6a). 175.0M parameters, 89.2M trainable. The ViT in the authors' checkpoint equals timm [13] vit_base_patch16_224, so its features are cached. Loss 5 × Chamfer; selection by val CD; configuration in Table 6.

Training. 50 epochs × 213 s; best val CD at epoch 40. Train loss keeps falling (8.1 → 3.0) while val CD is flat after epoch 30: the head starts to overfit (Figure 4). Inference 10.4 ms/image, 0.89 GB.

Table 4: Baseline 1: paper vs ours (CD / EMD ×100 / F@0.01)

Class	Paper [1]	Author weights, test_clean	Ours, test_clean	Ours, full test
airplane	2.73 / 5.01 / -	2.892 / 5.276 / 0.413	2.667 / 4.489 / 0.490	2.588 / 4.687 / 0.506
car	4.05 / 3.59 / -	4.361 / 3.830 / 0.085	4.073 / 3.449 / 0.097	4.003 / 3.370 / 0.101
chair	5.38 / 7.80 / -	5.591 / 7.800 / 0.074	5.506 / 7.755 / 0.078	5.363 / 7.903 / 0.084
mean	**4.05 / 5.47 / 0.505 (a)**	**4.281 / 5.635 / 0.190**	**4.082 / 5.231 / 0.221**	**3.985 / 5.320 / 0.230**

(a) Computed with 8192 points, not comparable with our 1024. Paper Pix3D CD 7.00; ours 6.545 on our protocol.

Gap analysis. CD/EMD are within 6% of the paper, and on test_clean our model beats the released weights (4.082 vs 4.281). Causes: (1) a different split (35-72% of our test objects are in the authors' training set); (2) chair has 10 views, not 24; (3) the official code differs from the paper (lr 5e-4, hidden 4096 instead of 1024, 1000 epochs, checkpoints chosen on test). We followed the paper's lr, 50 epochs and val selection.

2. Baseline 2: 3D-RETR-S

Model. A fine-tuned DeiT-Tiny (distilled) encodes the image into 198 tokens of 192-d. A 6-layer decoder (3 heads, dropout 0.4) with 64 queries, one per cell of a 4^3 grid, cross-attends to them; a 3D CNN (2 residual blocks, 3 transposed convolutions) upsamples to 32^3 logits (Figure 7a). 10.34M parameters. Loss Dice; selection by val IoU (Table 6). **Porting:** the PyTorch 1.8 / Lightning 1.3 code was rewritten in plain PyTorch with the same layers and preprocessing (fp16 → bf16).

Training. 50 epochs × 50 s (42 min); best val IoU 0.679 at epoch 49, still rising (Figure 5). Train IoU 0.710 vs test 0.678 indicates underfitting. Inference 17.9 ms/image, 0.08 GB.

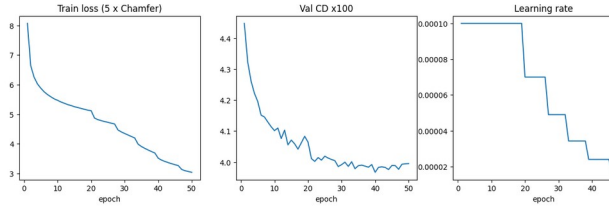


Figure 4: Baseline 1: train loss, val CD and learning rate.

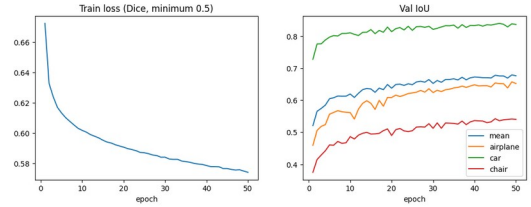


Figure 5: Baseline 2: Dice loss and val IoU per class.

Results. Test IoU, paper vs ours: airplane 0.696 vs 0.649, car 0.858 vs 0.841, chair 0.579 vs 0.543, mean 0.711 (3-class mean from [2]; 0.674 over 13 classes) vs 0.678 (train 0.710). **Gap analysis:** 0.016-0.047 lower IoU, mainly from much shorter training (42 min vs 1-3 days), then 10 chair views. Pix3D chair: paper (3D-RETR-B) 0.290, ours 0.293, not directly comparable (B vs S model; our IoU is approximate, GT points voxelised).

Table 5: Baselines, paper vs ours (means over airplane, car, chair)

Model	Source	CD ×100	EMD ×100	IoU	Dataset
RGB2Point	Paper [1]	4.05	5.47	-	ShapeNet, authors' split
RGB2Point	Ours	3.985	5.320	0.505 (b)	ShapeNet, 3D-RETR split
3D-RETR-S	Paper [2]	-	-	0.711 (c)	ShapeNet, R2N2 split
3D-RETR-S	Ours	4.552 (b)	-	0.678	ShapeNet, 3D-RETR split

(b) Converted to the other representation (Section VI.4). (c) Mean of the 3 classes from [2]; the paper reports 0.674 over 13 classes.

V. Proposed Method

1. Motivation from Error Analysis

Each version answers one finding of the baseline error analysis on all 69,034 test views (Section VI.5):

- **Single-token attention.** The CFI of B1 attends over one token, i.e. a product of Linear layers (about 67M redundant parameters) → V0 removes it.
- **Object, not view.** Between-object std is 2.4-4.2× the within-object std → improve shape detail, not viewpoint robustness.
- **Mean-shape collapse.** CD between two predictions is only 60-78% of that between two GTs → V1 lets the decoder read spatial patch tokens; V2 gives it better local features.
- **Density floor.** The 1024-point floor is already reached for car (4.49) → judge progress on chair and airplane.
- **Background attention.** Deep layers of B1-V1 attend to background (0.3% on the object) → V2: DINOv2 with registers.
- **Underfitting (B2).** Val IoU still rising at epoch 50 → B2-V3: cosine schedule.
- **Over-filling (B2).** Chair precision 0.662 < recall 0.735, thin chairs worst → B2-V1/V2: better encoder, finer queries.
- **Real photos.** Large drop on Pix3D for both → B1-V3 / B2-V4: augmentation and partial fine-tuning.

2. Point Cloud Line (Baseline 1 → V0-V3)

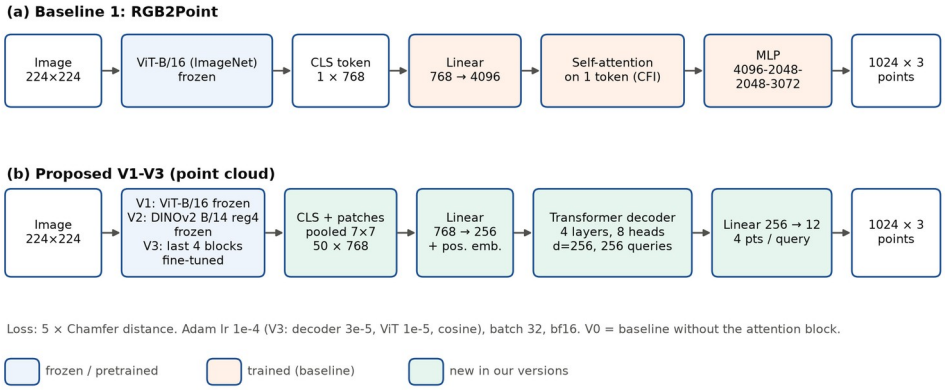


Figure 6: (a) RGB2Point baseline. (b) Our point cloud model: CLS + pooled patch tokens, a query-based Transformer decoder, and a DINOv2 encoder (V2 frozen; V3 last 4 blocks fine-tuned).

- **V0, no single-token attention.** Attention over one token is linear, so the CFI only adds parameters. V0 keeps Linear + MLP (22.0M trainable instead of 89.2M).
- **V1, query decoder over patch tokens.** CLS plus the 14 × 14 patch grid average-pooled to 7 × 7 give T = 50 tokens, projected to d = 256 with a learned position embedding (memory M). K = 256 learned queries pass through a 4-layer pre-norm decoder (8 heads, FFN 1024; self-attention, cross-attention to M, FFN); a Linear layer maps each query to 4 points. Each query can specialise on one part and read only the regions it needs. Trainable: 4.5M.
- **V2, DINOv2 encoder.** V1's deep layers attend to background (Section VI.5.4): last-layer patch tokens of a classification ViT carry global information. V2 uses a frozen DINOv2 ViT-B/14 reg4 with the same pooling and decoder.
- **V3, partial fine-tuning.** From V2, the last 4 of 12 blocks and the final norm are fine-tuned (lr 1e-5, decoder 3e-5), cosine schedule, 8 epochs, with augmentation. Goal: real-photo robustness.

The decoder head shared by V1-V3:

$$M = W_{\text{in}}\,[\text{CLS};\ \text{Pool}_{7\times 7}(\text{patches})] + E_{\text{pos}} \tag{4}$$

$$Z = \text{Decoder}(Q_0, M), \qquad \hat{P} = \text{reshape}(W_{\text{out}} Z) \in \mathbb{R}^{1024\times 3} \tag{5}$$

3. Voxel Line (Baseline 2 → V1-V4)

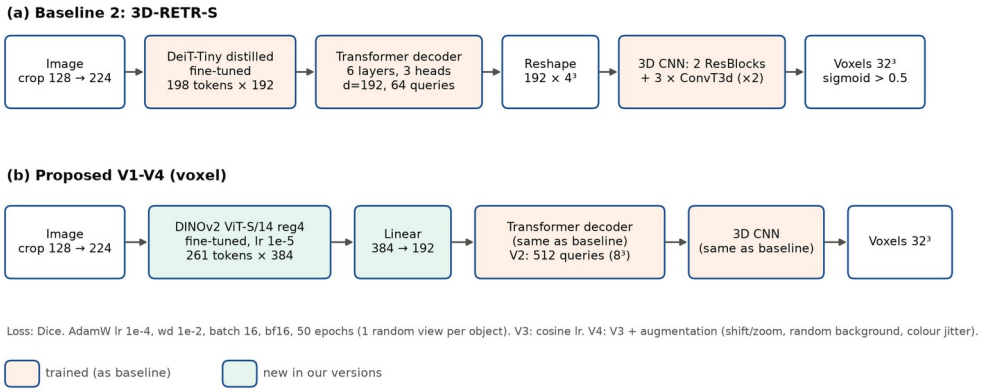


Figure 7: (a) 3D-RETR-S baseline. (b) Our voxel model: DINOv2 ViT-S/14 (reg4) encoder with a 384 →192 projection; decoder and 3D CNN unchanged (V2: 8^3 query grid).

- **V1, DINOv2 encoder:** ViT-S/14 reg4 (22M, 261 tokens of 384-d) replaces DeiT-Tiny, with a 384 → 192 projection so the decoder is unchanged; encoder lr 1e-5 (10× smaller). Motivated by B1-V2.
- **V2, finer query grid:** 8^3 = 512 queries, each for a 4^3 block. Motivated by thin chair parts.
- **V3, cosine schedule:** V1 with cosine decay over 50 epochs. Motivated by underfitting.
- **V4, augmentation:** V3 plus augmentation (Section III.5). Motivated by the real-photo gap.

4. Losses and Training Strategy

$$\mathcal{L}_{\text{CD}} = \alpha \left[\sum_{p \in P} \min_{q \in Q} \|p - q\|_2^2 + \sum_{q \in Q} \min_{p \in P} \|q - p\|_2^2 \right], \qquad \alpha = 5 \tag{6}$$

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{\sum_i p_i t_i + 1}{\sum_i (p_i^2 + t_i^2) + 1}, \qquad p_i = \sigma(\text{logit}_i), \;\; t_i \in \{0, 1\} \tag{7}$$

- One change per version, everything else fixed, so each ablation row measures one idea.
- Same seed, split, epochs (50; 8 for fine-tuning) and selection rule (best val metric) for all versions.
- Frozen encoders are run once and cached; fine-tuned encoders use a 3-10× smaller lr than the head.
- A version that does not improve on val is not built upon (B2-V2 dropped; V3 and V4 build on V1).

VI. Experiments

1. Settings and Hyper-parameters

Google Colab Pro, NVIDIA L4 24 GB, High-RAM; Python 3.13, PyTorch 2.11 (CUDA 13.0), torchvision 0.26, timm 1.0.30; bf16; seed 0. Times are per epoch including validation.

Table 6: Hyper-parameters of every run

Run	Encoder (trainable?)	Head	Optimiser, lr	Schedule	Batch, epochs	Loss	Time / epoch	Best epoch
B1 base	ViT-B/16 (frozen)	Linear + CFI + MLP	Adam 1e-4, clip 5	Plateau 0.7/5	32, 50	5 × CD	213 s	40
B1-V0	ViT-B/16 (frozen)	Linear + MLP	Adam 1e-4, clip 5	Plateau 0.7/5	32, 50	5 × CD	100 s	38
B1-V1	ViT-B/16 (frozen)	Query decoder	Adam 1e-4, clip 5	Plateau 0.7/5	32, 50	5 × CD	213 s	38
B1-V2	DINOv2 B/14 reg4 (frozen)	Query decoder	Adam 1e-4, clip 5	Plateau 0.7/5	32, 50	5 × CD	220 s	50
B1-V3	DINOv2 B/14, last 4 blocks	Query decoder	Adam 3e-5 head, 1e-5 ViT	Cosine	32, 8 (from V2)	5 × CD	938 s	7
B2 base	DeiT-Tiny (yes)	Decoder 64 q. + 3D CNN	AdamW 1e-4, wd 1e-2	Constant	16, 50	Dice	50 s	49
B2-V1	DINOv2 S/14 reg4 (lr 1e-5)	Decoder 64 q. + 3D CNN	AdamW 1e-4, wd 1e-2	Constant	16, 50	Dice	54 s	50
B2-V2	DINOv2 S/14 reg4 (lr 1e-5)	Decoder 512 q. + 3D CNN	AdamW 1e-4, wd 1e-2	Constant	16, 50	Dice	56 s	49
B2-V3	DINOv2 S/14 reg4 (lr 1e-5)	Decoder 64 q. + 3D CNN	AdamW 1e-4, wd 1e-2	Cosine	16, 50	Dice	81 s	50
B2-V4	DINOv2 S/14 reg4 (lr 1e-5)	Decoder 64 q. + 3D CNN	AdamW 1e-4, wd 1e-2	Cosine + aug.	16, 50	Dice	81 s	50

Fixed: B1 decoder dim 256, 4 layers, 8 heads, 256 queries × 4 points; B2 decoder 6 layers, 3 heads, dim 192, dropout 0.4, voxel threshold 0.5. B1 epochs use all views, B2 epochs one random view per object. Configs: configs/*.yaml.

2. Main Results

Table 7: Point cloud line, full test set (CD / EMD ×100 / F@0.01; view 00, 3668 objects; CD, EMD, F = mean over classes; Train = trainable parameters)

Version	airplane	car	chair	CD	EMD	F	Train (M)	ms/ img	Pix3D CD / F
Baseline	2.588 / 4.687 / 0.506	4.003 / 3.370 / 0.101	5.363 / 7.903 / 0.084	3.985	5.320	0.230	89.2	10.4	6.545 / 0.080
V0	2.608 / 4.052 / 0.507	4.028 / 3.375 / 0.100	5.412 / 6.355 / 0.087	4.016	4.594	0.231	22.0	8.4	-
V1	2.495 / 4.225 / 0.526	3.856 / 3.204 / 0.110	4.906 / 5.926 / 0.099	3.752	4.452	0.245	4.5	13.0	6.272 / 0.092
V2	2.282 / 3.391 / 0.581	3.694 / 3.062 / 0.122	4.424 / 5.477 / 0.119	3.467	3.977	**0.274**	4.5	13.6	5.761 / 0.106
V3	**2.262** / **3.324** / 0.575	**3.651** / 3.063 / **0.126**	**4.360** / **5.266** / 0.119	**3.424**	**3.884**	0.273	32.9	14.4	**5.486** / **0.111**

On test_clean (1664 images) the ranking is the same, mean CD / EMD / F: baseline 4.082 / 5.231 / 0.221, V0 4.129 / 4.576 / 0.223, V1 3.874 / 4.445 / 0.235, V2 3.582 / 4.019 / 0.261, V3 3.550 / 3.949 / 0.260. Total parameters 175.0M, 107.8M, 90.3M, 90.2M, 90.2M; GPU memory 0.89, 0.49, 0.38, 0.38, 0.44 GB.

Table 8: Voxel line, full test set (IoU; view 00, 3668 objects)

Version	airplane	car	chair	Mean IoU	Params (M)	ms/img	Pix3D IoU	Pix3D CD / F
Baseline	0.649	0.841	0.543	0.678	10.34	17.9	0.293	7.391 / 0.071
V1	0.657	0.850	0.552	0.686	26.52	19.3	0.311	**7.051** / **0.076**
V2	0.654	0.850	0.553	0.686	26.34	18.9	0.302	7.365 / 0.072
V3	**0.660**	**0.853**	**0.562**	**0.692**	26.52	19.2	0.304	7.249 / 0.074
V4	0.625	0.832	0.536	0.664	26.52	18.6	**0.327**	7.112 / 0.075

GPU memory 0.08 GB (baseline), 0.18 GB (V1-V4). Pix3D IoU is approximate: the point GT is voxelised.

3. Ablation Study

Table 9: Ablation: one change per version (CD ×100 lower is better, IoU higher is better)

Version	Change	Reason	Metric	Result	Comment
B1-V0	Remove single-token attention (CFI)	Attention on 1 token is linear	Mean CD / EMD	4.016 vs 3.985 / 4.594 vs 5.320	Same CD, 1/4 trainable params; chair EMD 7.90 → 6.36
B1-V1	Query decoder on CLS + 7×7 patches	Spatial features unused; mean-shape collapse	Mean CD	3.752 (-5.8%)	Chair CD -8.5%, EMD -25%; 4.5M trainable
B1-V2	DINOv2 ViT-B/14 reg4 (frozen)	V1 patch tokens not local	Mean CD	3.467 (-7.6% vs V1)	Better than V1 on about 92% of objects
B1-V3	Fine-tune last 4 blocks + aug.	Real-photo gap	Pix3D CD	5.486 (vs 5.761)	Small ShapeNet gain (3.424)
B2-V1	DINOv2 ViT-S/14 reg4	Lesson from B1-V2	Mean IoU	0.686 (+0.008)	Less over-filling; train IoU 0.721
B2-V2	512 queries (8^3)	Thin chair parts	Mean IoU	0.686 (=V1)	No gain; dropped
B2-V3	Cosine lr (from V1)	Not converged	Mean IoU	0.692 (+0.014)	Best on ShapeNet
B2-V4	V3 + augmentation	Real-photo gap	Pix3D IoU	0.327 (+0.034)	Best on Pix3D; -0.028 ShapeNet

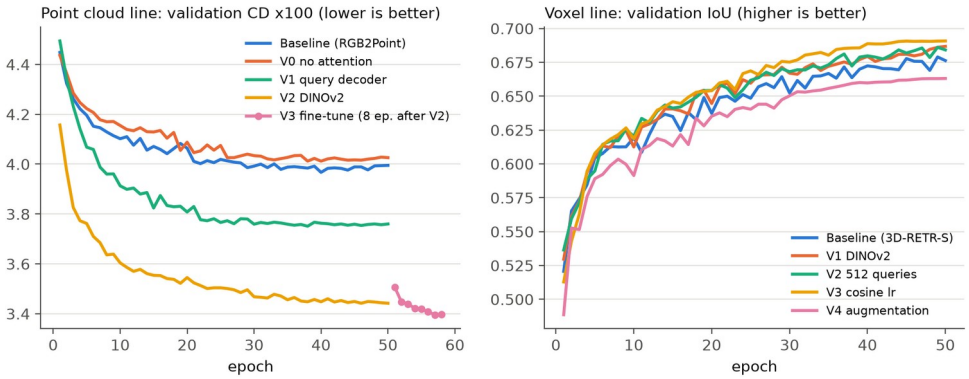


Figure 8: Validation curves of every version: point cloud line (left; V3 continues from V2 for 8 epochs) and voxel line (right).

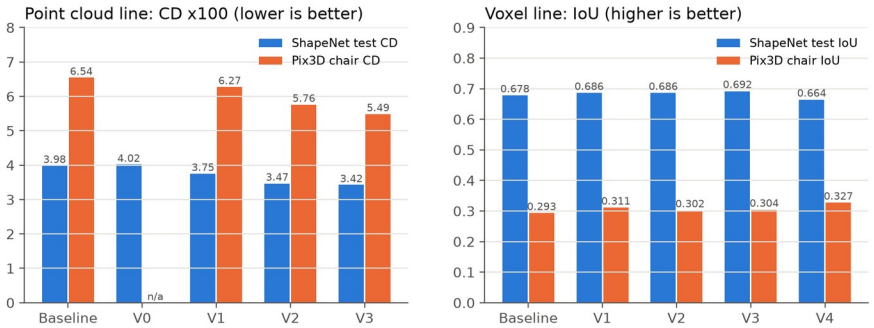


Figure 9: Ablation on ShapeNet test and Pix3D chair: point cloud line CD (left), voxel line IoU (right). V0 was not evaluated on Pix3D.

- **Point clouds:** V0 shows the CFI is unnecessary; the largest gains come from spatial tokens (V1) and local features (V2).
- **Voxels:** the encoder helps less (+0.008) because the model underfits (V1 train IoU 0.721); the cosine schedule (V3) gives the best ShapeNet IoU.
- **Augmentation** trades synthetic for real-photo accuracy (V4 vs V3: -0.028 ShapeNet, +0.023 Pix3D IoU). The demo uses B2-V4, a choice made on Pix3D, so Pix3D is not fully held-out for V4.

4. Cross-Representation Comparison

Voxel outputs are converted to 1024 surface points (diagonal 1, GT-to-GT rotation) and point outputs to voxels (R2N2 placement, 5 closings chosen on val GT, interior filling). The GT converted the same way gives the ceiling.

Table 10: All models on common metrics (test view 00, means over 3 classes)

Model	Mean IoU	Mean CD ×100	Mean F@0.01	Chair IoU	Chair CD
Ceiling: GT converted	**0.723**	**3.664**	**0.197**	**0.602**	**3.941**
RGB2Point (B1 base)	0.505	3.985	0.230	0.316	5.363
B1-V0	0.508	4.016	0.231	0.325	5.412
B1-V1	0.539	3.752	0.245	0.371	4.906
B1-V2	0.574	3.467	**0.274**	0.411	4.424
B1-V3	0.578	**3.424**	0.273	0.425	**4.360**
3D-RETR-S (B2 base)	0.678	4.552	0.158	0.543	5.633
B2-V1	0.686	4.476	0.160	0.552	5.502
B2-V2	0.686	4.530	0.159	0.553	5.607
B2-V3	**0.692**	4.559	0.157	**0.562**	5.653
B2-V4	0.664	4.664	0.152	0.536	5.720

Each representation wins on its own metric (points on CD and F, voxels on IoU), partly because of the conversion ceiling (IoU $\leq$ 0.723; F $\leq$ 0.197 from 1/32 quantisation). The ranking of versions within each line is mostly preserved, which supports the ablation.

5. Error Analysis

5.1 Viewpoint vs object

Over all 69,034 test views, viewpoint matters little: B1 chair CD ranges 5.24-5.55 over 8 azimuth bins and B2 chair IoU 0.52-0.56. The std of per-object mean CD (airplane 1.01, car 0.58, chair 1.47) far exceeds the std between views of one object (0.24, 0.21, 0.61); for IoU about 3× (0.169/0.089/0.160 vs 0.051/0.026/0.058). Error is a property of the object.

5.2 Mean-shape collapse and density floor

Table 11: Diversity of predictions: metric between two different objects (higher CD / lower IoU = more diverse)

Class	B1 CD: base / V1 / V2 / GT	B2 IoU: pred. / GT	Class-mean template IoU	1024-pt floor CD / F
airplane	3.06 / 3.41 / 3.92 / 5.10	0.354 / 0.300	0.433	1.43 / 0.768
car	4.26 / 4.50 / 4.76 / 5.47	0.710 / 0.625	0.725	4.49 / 0.167
chair	7.67 / 9.31 / 9.92 / 11.21	0.169 / 0.130	0.235	2.56 / 0.350

Two predicted chairs are closer to each other than their GTs (baseline CD ratio 0.68), the retrieval behaviour of [11]; V1 and V2 raise the ratio to 0.83 and 0.89. A fixed class-mean template already reaches car IoU 0.725, and the car density floor (CD between two 1024-point samples of one GT, 4.49) exceeds the baseline CD (4.00), so the room for improvement lies in chair and airplane.

5.3 Per-object comparison

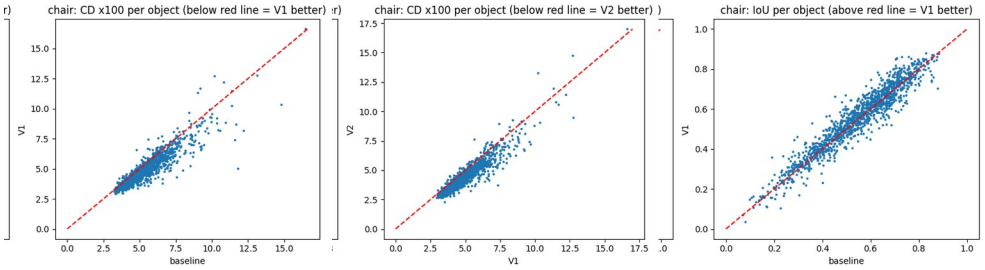


Figure 10: Per-object results on test chairs (mean over views): B1 base vs V1 CD, V1 vs V2 CD, B2 base vs V1 IoU. Points below (CD) or above (IoU) the red line favour the newer version.

B1-V1 beats the baseline on 74.2% / 82.3% / 85.3% of airplanes / cars / chairs (median CD -2.5%, -2.9%, -7.1%); on the hardest 10% chairs CD drops 8.54 → 7.56. B1-V2 beats V1 on 92.7% / 91.8% / 92.1% of objects (hardest chairs 6.77). B2-V1 beats its baseline on 63.7% / 68.5% / 60.4% of objects.

5.4 Attention analysis of B1-V1

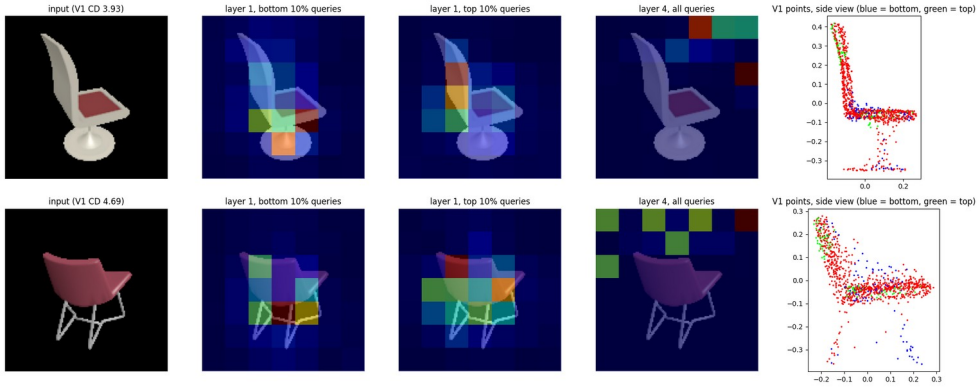


Figure 11: B1-V1 cross-attention on two test chairs. Layer 1: queries of the lowest 10% of points look at legs and base, the top 10% at the backrest. Layer 4: all queries look at the same background cells.

The object covers 15.1% of the image. In V1 the share of patch attention on object cells falls from 0.372 (layer 1) to 0.121, 0.005 and 0.003; in V2 it is 0.365, 0.147, 0.106 and 0.235. V1's first layer is part-aware, but layers 3-4 put 99.5% of their attention on background and all queries agree (cosine 0.998). These tokens are not high-norm artefacts [10]; the late tokens of a classification ViT simply carry global information. DINOv2 restores attention to the object (0.003 → 0.235), consistent with the CD gain.

5.5 Voxel errors and best/worst cases

Table 12: Baseline 2 by GT size quartile (Q0 = thinnest objects); IoU / precision / recall

Class	Q0	Q1	Q2	Q3	Pred. vs GT voxels
airplane	0.598 / 0.720 / 0.752	0.685 / 0.806 / 0.802	0.705 / 0.831 / 0.807	0.601 / 0.764 / 0.721	655 vs 659
car	0.805 / 0.860 / 0.924	0.870 / 0.921 / 0.939	0.873 / 0.940 / 0.922	0.821 / 0.929 / 0.871	2993 vs 3011
chair	0.471 / 0.576 / 0.696	0.532 / 0.652 / 0.723	0.536 / 0.662 / 0.720	0.637 / 0.758 / 0.799	4191 vs 3802

Chairs are over-filled (precision 0.662 < recall 0.735) and the thinnest are hardest (IoU 0.471). B2-V1 reduces over-filling (precision 0.685) but thin chairs stay at 0.478, and 512 queries (V2) did not help: the 32^3 resolution and Dice loss, not the query count, limit thin parts. Threshold 0.4 vs 0.5 changes val IoU by 0.001.

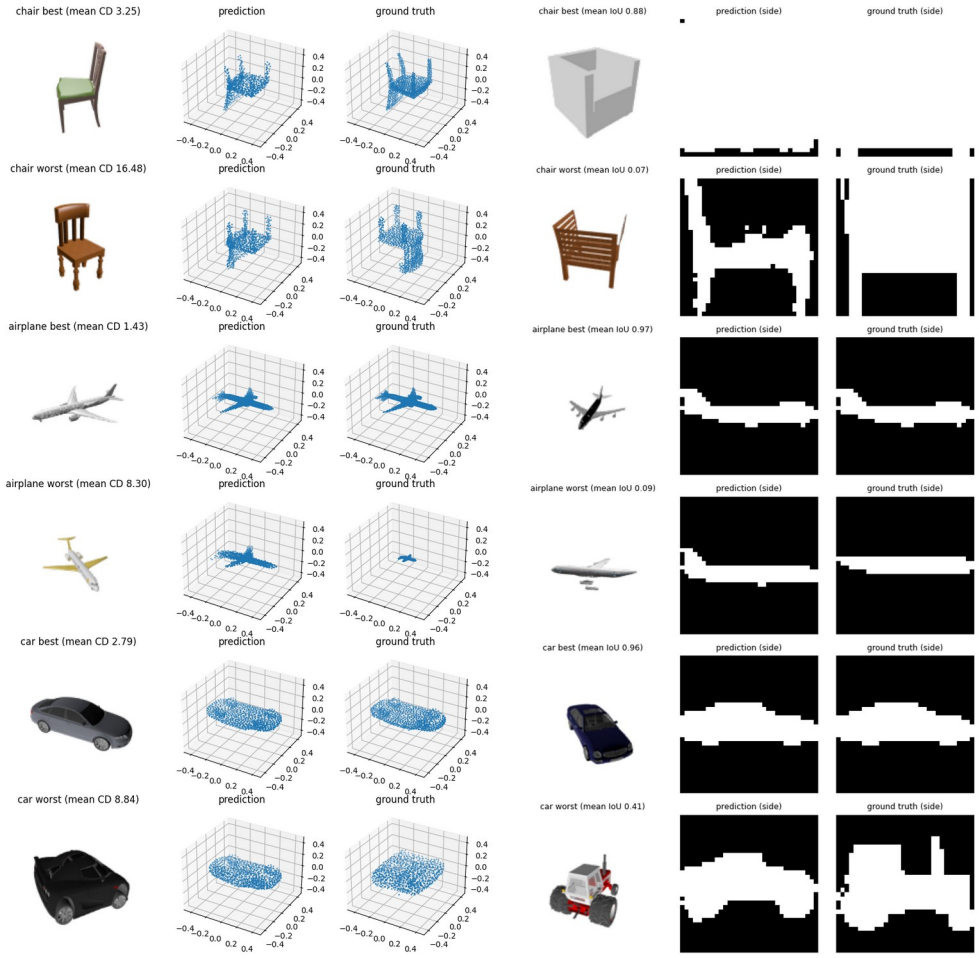


Figure 12: Best and worst test objects per class. Left, Baseline 1: the worst airplane has an abnormally small GT, the worst chair thin legs. Right, Baseline 2: the worst chair is a mis-oriented GT, the worst car a tractor.

6. Qualitative Results on Real Photos

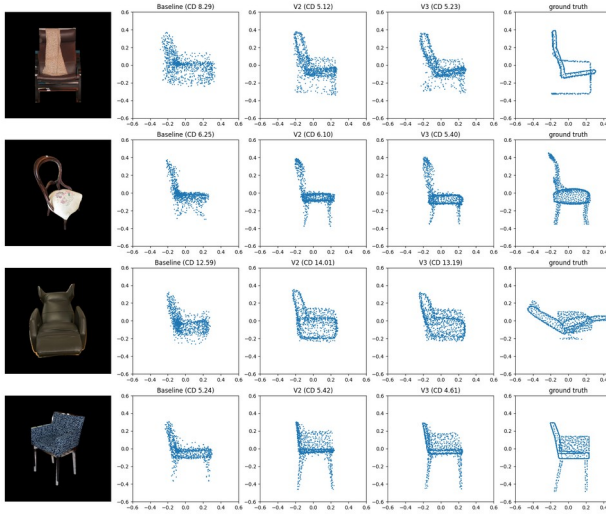


Figure 13: Pix3D chairs, point cloud line: baseline, V2, V3, GT. V3 recovers legs and backrest better; the reclining chair (row 3) fails for all.

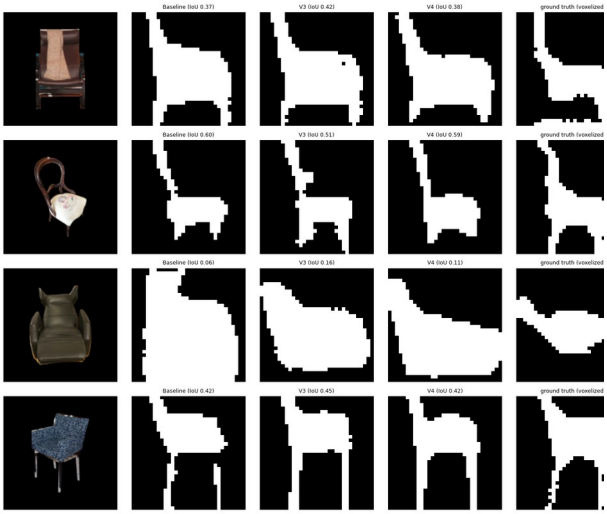


Figure 14: Pix3D chairs, voxel line (side projection): baseline, V3, V4, voxelised GT.

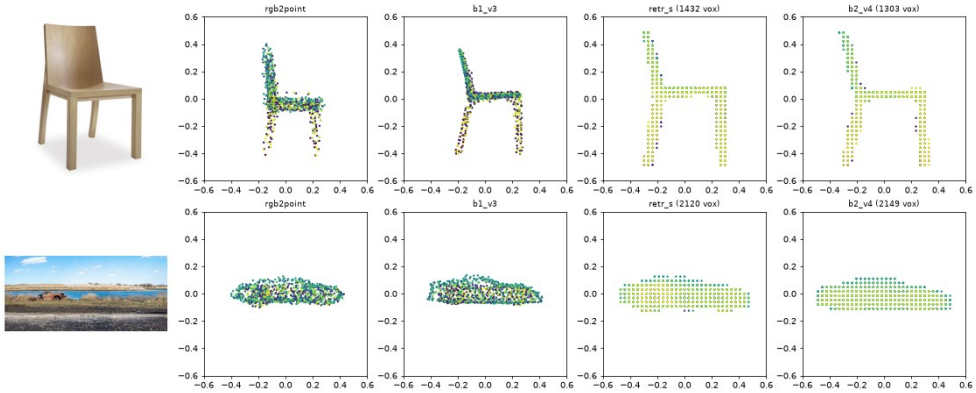


Figure 15: Internet photos (Wikimedia Commons, test only) through the demo pipeline, 4 models. B1-V3 gives clearer chair legs and backrest than RGB2Point. Photos: "Library Chair dakota jackson.jpg" by Dakota Jackson Inc., CC BY-SA 3.0, commons.wikimedia.org/wiki/File:Library_Chair_dakota_jackson.jpg; "Abandoned car in Marine Park (10852p).jpg" by Rhododendrites, CC BY-SA 4.0, commons.wikimedia.org/wiki/File:Abandoned_car_in_Marine_Park_(10852p).jpg.

7. Efficiency

Table 13: Size and speed of the models used in the demo

Model	Total params	GPU ms/img (L4)	GPU memory	CPU ms/img (laptop)	fp16 weights
RGB2Point (rgb2point)	175.0M	10.4	0.89 GB	-	350 MB
B1-V3 (b1_v3)	90.2M	14.4	0.44 GB	85-110	181 MB
3D-RETR-S (retr_s)	10.3M	17.9	0.08 GB	20-60	21 MB
B2-V4 (b2_v4)	26.5M	18.6	0.18 GB	20-60	53 MB

All models need under 1 GB of GPU memory. On Cloud Run (2 vCPU) b1_v3 takes about 400 ms and a full request 2-3 s, background removal included. fp16 export changes points by at most 0.0005 and keeps 99.99% of voxels, so the models can be served on CPU (Section VII). Timings are from one measurement run; re-running on the same L4 changes them by up to about 1 ms per image (the executed notebooks show 17.0-18.4 ms for the voxel line and 8.8-14.8 ms for the point line), so differences below 1 ms are not meaningful.

VII. Demo and Deployment

1. Application Architecture

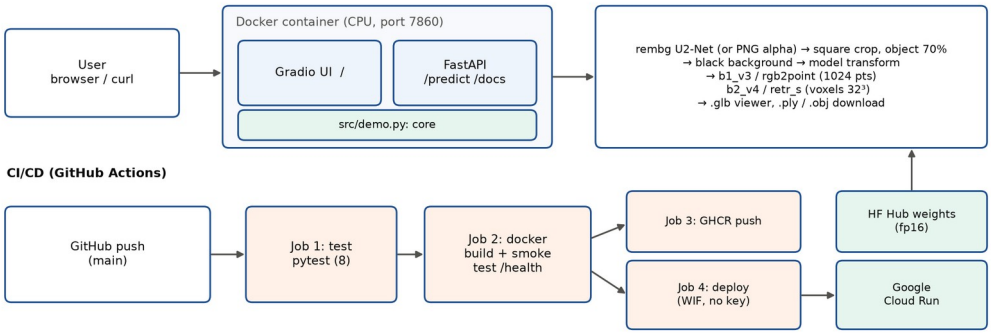


Figure 16: Application and CI/CD architecture. One container serves the Gradio demo (/) and the REST API (/predict, /docs); weights come from the Hugging Face Hub on first use. HF Hub only hosts the weights; the app runs on Cloud Run.

A FastAPI [16] server (api/main.py) mounts the Gradio [15] app (app/gradio_app.py) at /. Both call one core module (src/demo.py) for preprocessing, prediction and export, so demo and API give identical results. Four models: b1_v3 and rgb2point (points), b2_v4 and retr_s (voxels), on CPU.

2. Inference Pipeline

- **Background removal:** alpha channel if present, else rembg [14] with U2-Net [12] (class-agnostic; a DeepLabV3 VOC mask was empty on clear chair photos).
- **Crop and transform:** square crop with the object filling 70% of the side, black background, 224 × 224, then the training transform of each model.
- **Output:** points or voxels (sigmoid > 0.5) rotated to y-up and scaled to diagonal 1; .glb for the viewer, .ply (points) and .obj (voxel mesh) for download.

3. User Guide

(1) Upload or take a photo, or pick one of 24 test objects; (2) choose a point cloud and a voxel model; (3) Reconstruct, then rotate, zoom and download. Test objects also show the GT and CD, F@0.01 and IoU, matching the evaluation pipeline within 0.001. Light and dark themes; works on mobile.

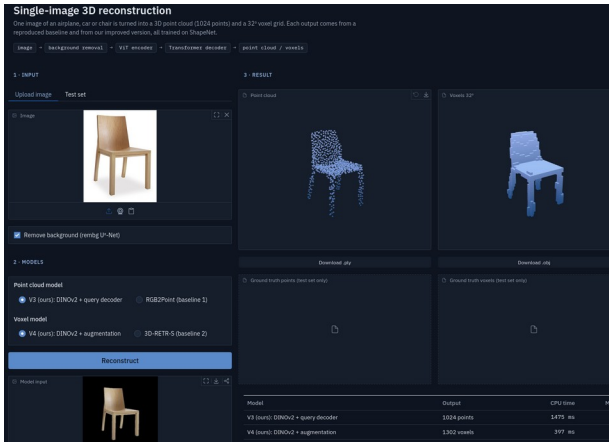


Figure 17: Upload image tab: a chair photo, the model input after background removal, B1-V3 points and B2-V4 voxels. Photo: "Library Chair dakota jackson.jpg" by Dakota Jackson Inc., CC BY-SA 3.0, commons.wikimedia.org/wiki/File:Library_Chair_dakota_jackson.jpg.

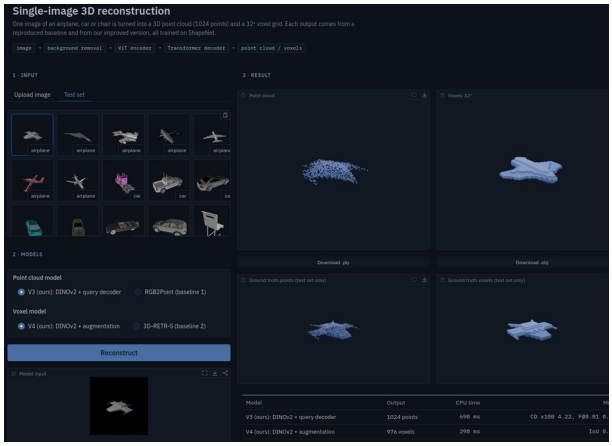


Figure 18: Test set tab: a test airplane with its ground truth and per-object metrics (CD, F@0.01, IoU) next to the predictions.

4. REST API (FastAPI)

- **GET /health**, **GET /models:** status and the list of models (name, label, output type).

- **POST /predict:** multipart image; query model (b1_v3, rgb2point, b2_v4, retr_s), remove_background, format (json: points or voxels with timing; file: .ply / .obj).
- **GET /docs:** Swagger UI with schemas and error codes 400 (not an image), 422 (bad parameter), 503 (weights not loaded).

Example: curl -F image=@chair.jpg "<host>/predict?model=b2_v4&format=file" -o chair.obj.



Figure 19: Uploaded photos after background removal and cropping. Photos: Dakota Jackson Inc. (CC BY-SA 3.0) and Rhododendrites (CC BY-SA 4.0), Wikimedia Commons; full credits in Figure 15.

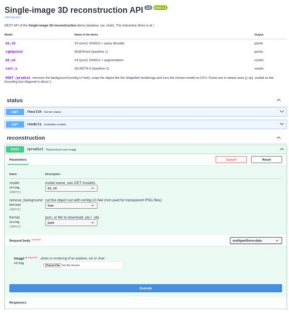


Figure 20: Swagger page of the API (/docs).

5. Docker, CI/CD and Cloud Deployment

- **Docker:** python:3.12-slim, CPU PyTorch 2.11, pinned requirements-demo.txt, non-root user, U2-Net fetched at build time; about 2.8 GB. docker compose up --build runs it with a weight-cache volume.
- **CI/CD (GitHub Actions):** (1) test: 8 pytest tests with tiny random models; (2) docker: build and smoke-test /health; then on main, in parallel, (3) ghcr: push ghcr.io/ilogicus/image-to-3d-transformer:latest and (4) deploy to Google Cloud Run via Artifact Registry.
- **Cloud Run:** 8 GB RAM, 2 vCPU, scales to zero (first request about 1 min); login by Workload Identity Federation, no stored key. Chosen because HF Docker Spaces now need a paid plan and free Render/Koyeb tiers give 512 MB, below the 3 GB needed.
- **Weights:** fp16 on the Hugging Face Hub (about 600 MB), downloaded on first use.

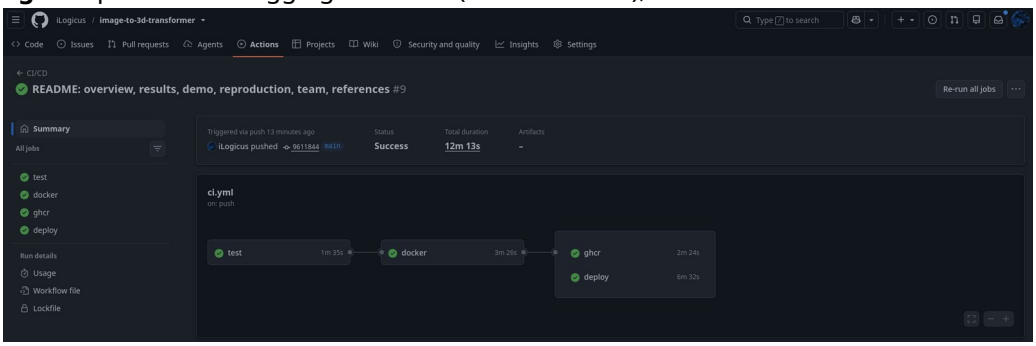


Figure 21: GitHub Actions run of the README commit: the four jobs (test, docker, ghcr, deploy) passed.

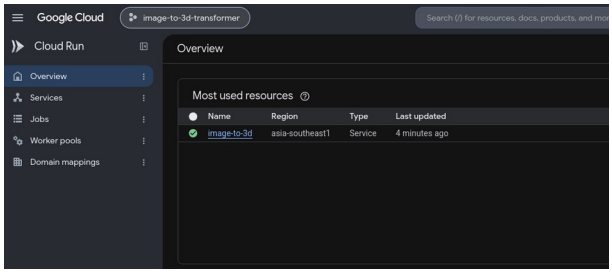


Figure 22: Cloud Run: the image-to-3d service in asia-southeast1.

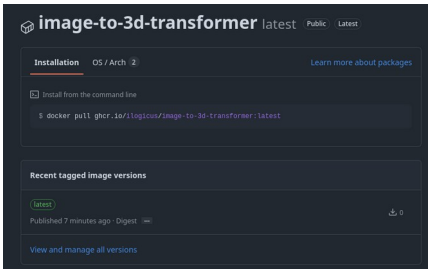


Figure 23: GHCR: the public image ghcr.io/ilogicus/image-to-3d-transformer:latest.

VIII. Conclusion and Future Work

1. Summary of Findings

- Both baselines were reproduced on a leak-free object split: RGB2Point within 6% of the paper (and better than the released weights on test_clean), 3D-RETR-S within 0.016-0.047 IoU after 42 minutes of training.
- Errors depend on the object rather than the viewpoint, and both baselines drift towards the class-mean shape.
- Point clouds: a query decoder over patch tokens (V1) and DINOv2 features (V2) cut mean CD by 13% with 20× fewer trainable parameters; partial fine-tuning with augmentation (V3) mainly helps real photos (Pix3D CD -16%).
- Voxels: DINOv2 and a cosine schedule give +0.014 IoU; augmentation gives the best Pix3D IoU (0.327 vs 0.293).
- All models run in real time on GPU and in about 0.1 s on CPU, and are deployed as a public web app and API.

2. Strengths and Weaknesses

- **Point cloud (B1-V3):** best CD and F, small trainable head, object-specific shapes, best Pix3D CD; but no surface, F limited by the 1024-point density, thin parts blurred.
- **Voxel (B2-V3 / V4):** best IoU, small (26.5M), fast on CPU, watertight output; but 32^3 resolution, over-filled thin chairs, and augmentation lowers ShapeNet IoU.

3. Limitations

- Three classes, low-resolution outputs and short training (50 epochs); B2 models are not fully converged.
- Single seed: no confidence intervals; small differences such as B2-V1 vs its baseline (+0.008 IoU) may be within run-to-run noise.
- Pix3D covers chairs only, its IoU is approximate, and it influenced the choice of the demo voxel model.
- Chair has 10 views and some mis-oriented GT; the demo fails on cluttered scenes or several objects.

4. Future Work

- Longer training with several seeds; all 13 ShapeNet classes.
- Denser outputs (2048-8192 points) or implicit surfaces to remove the density and resolution floors.
- Fine-tune DINOv2 in the voxel line with stronger augmentation to close the real-photo gap.
- Part-aware losses for thin structures, or generative (flow / diffusion) decoders.
- Collect real photos of the team's own objects for all three classes and evaluate on them.
- A GPU deployment option for lower latency.

IX. References

[1] J. J. Lee and B. Benes, "RGB2Point: 3D point cloud generation from single RGB images," in Proc. IEEE/CVF Winter Conf. Applications of Computer Vision (WACV), 2025. arXiv:2407.14979. Code: github.com/JaeLee18/RGB2point.

[2] Z. Shi, Z. Meng, Y. Xing, Y. Ma, and R. Wattenhofer, "3D-RETR: End-to-end single and multi-view 3D reconstruction with transformers," in Proc. British Machine Vision Conf. (BMVC), 2021. arXiv:2110.08861. Code: github.com/FomalhautB/3D-RETR.

[3] A. X. Chang et al., "ShapeNet: An information-rich 3D model repository," arXiv:1512.03012, 2015.

[4] C. B. Choy, D. Xu, J. Gwak, K. Chen, and S. Savarese, "3D-R2N2: A unified approach for single and multi-view 3D object reconstruction," in Proc. European Conf. Computer Vision (ECCV), 2016. arXiv:1604.00449.

[5] P. Mandikal, K. L. Navaneet, M. Agarwal, and R. V. Babu, "3D-LMNet: Latent embedding matching for accurate and diverse 3D point cloud reconstruction from a single image," in Proc. BMVC, 2018. arXiv:1807.07796.

[6] X. Sun et al., "Pix3D: Dataset and methods for single-image 3D shape modeling," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2018. arXiv:1804.04610.

[7] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in Proc. Int. Conf. Learning Representations (ICLR), 2021. arXiv:2010.11929.

[8] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jegou, "Training data-efficient image transformers & distillation through attention," in Proc. Int. Conf. Machine Learning (ICML), 2021. arXiv:2012.12877.

[9] M. Oquab et al., "DINOv2: Learning robust visual features without supervision," Transactions on Machine Learning Research, 2024. arXiv:2304.07193.

[10] T. Darcet, M. Oquab, J. Mairal, and P. Bojanowski, "Vision transformers need registers," in Proc. ICLR, 2024. arXiv:2309.16588.

[11] M. Tatarchenko, S. R. Richter, R. Ranftl, Z. Li, V. Koltun, and T. Brox, "What do single-view 3D reconstruction networks learn?" in Proc. CVPR, 2019. arXiv:1905.03678.

[12] X. Qin, Z. Zhang, C. Huang, M. Dehghan, O. R. Zaiane, and M. Jagersand, "U2-Net: Going deeper with nested U-structure for salient object detection," Pattern Recognition, vol. 106, 2020. arXiv:2005.09007.

[13] R. Wightman, "PyTorch Image Models (timm)," GitHub repository, 2019. github.com/huggingface/pytorch-image-models.

[14] D. Gatis, "rembg: a tool to remove image backgrounds," GitHub repository, MIT license. github.com/danielgatis/rembg.

[15] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan, and J. Zou, "Gradio: Hassle-free sharing and testing of ML models in the wild," arXiv:1906.02569, 2019.

[16] S. Ramírez, "FastAPI," fastapi.tiangolo.com.

X. Appendices

A. Task Assignment

Table 14: Task assignment

Member	Tasks	Deliverables	Evidence
Đỗ Nguyễn Hữu Phúc (SE204297)	Data pipeline; reproduction of both baselines; error analysis; B1 V0-V3 and B2 V1-V4; cross-representation comparison; Pix3D evaluation; Gradio demo, FastAPI, Docker, CI/CD, Cloud Run deployment; writing the report (AI-assisted, Appendix B)	src/, configs/, the two notebooks, app/, api/, docker/, .github/, report	Commit history, GitHub Actions
Nguyễn Tiến Vinh (SE204328)	Report review: numbers checked against the notebooks, spelling, citations; screenshots of the deployment (GitHub Actions 4 jobs, Cloud Run, GHCR) and of the interface (Section VII)	List of corrections, screenshots	Co-authored commit when the screenshots are added
Phạm Trường Minh Nhật (SE205274)	Slides; demo video (2-5 min); acceptance test as the lecturer (clean clone, docker compose, CLI)	Slide file, video and link, test notes	Co-authored commit when the slides and video are added
All members	Oral exam preparation: each member explains their own part and the whole project	-	-

The work of members 2 and 3 is committed by member 1 with a Co-authored-by line, so that GitHub credits both authors. All members used AI tools, as declared in Appendix B.

B. Use of AI Tools

All three members used Claude (Anthropic) and confirm this declaration. Member 1 used it through the Claude Code assistant during the whole project: writing and reviewing code (data pipeline, model re-implementations, porting 3D-RETR from PyTorch 1.8 / Lightning 1.3 to plain PyTorch, evaluation and analysis scripts, demo, API, Docker and CI/CD), debugging, and drafting this report and its figures from the notebook outputs. All experiments were run by the team on Colab; every number in this report comes from the executed notebooks; design decisions were made and checked by the team, and each member must be able to explain every line of the code in the oral exam. Member 2 used Claude when reviewing the report and preparing the deployment and interface screenshots; member 3 used it for the slides, the demo video and the acceptance test. All content was checked by the team.

C. Reproducibility

- **Code and environment:** github.com/iLogicus/image-to-3d-transformer (configs/, the two executed notebooks, requirements.txt); Python 3.13, torch 2.11 (cu130), torchvision 0.26, timm 1.0.30; seed 0.
- **Train / evaluate:** src/train.py, src/evaluate.py (points); src/train_retr.py, src/evaluate_retr.py (voxels); src/eval_pix3d.py, src/eval_pix3d_retr.py (Pix3D); src/cross_eval.py (common metrics); all with --config configs/<run>.yaml.
- **Demo:** docker compose up --build, or python api/main.py; tests: python -m pytest tests.
- **CLI:** python src/inference.py --image x.jpg --model b1_v3
- **Weights and data:** huggingface.co/iLogicus/image-to-3d-weights; Table 2 and the README.